

Centro Universitário Católica de Santa Catarina

Engenharia de Software



Adopt.me - Sistema web de adoção de animais

Ana Carolina Rocha

Joinville, SC

Resumo

Este documento descreve o desenvolvimento do sistema web Adopt.me, uma plataforma voltada para facilitar o processo de adoção responsável de animais. O sistema permite que usuários realizem cadastro, publiquem animais disponíveis para adoção e encontrem animais compatíveis com seu perfil por meio de um mecanismo de recomendação baseado em compatibilidade entre atributos previamente definidos. A aplicação foi desenvolvida utilizando React no frontend, Flask (Python) no backend e PostgreSQL como banco de dados principal. A lógica de recomendação foi implementada utilizando cálculo de similaridade entre perfis, considerando critérios como tipo de moradia, presença de crianças, tempo disponível e estilo de vida do adotante. O sistema conta ainda com autenticação via Google, autenticação baseada em JWT, filtros de busca e infraestrutura em produção utilizando Docker e AWS.

1. Introdução

Contexto:

A adoção de animais muitas vezes ocorre de forma desorganizada e sem critérios técnicos, o que pode levar à devolução de animais ou adoções mal-sucedidas.

Justificativa:

O projeto é relevante pois aplica princípios de engenharia de software em um contexto social, buscando incentivar adoções mais responsáveis e reduzir incompatibilidades entre adotantes e animais disponíveis.

Objetivos:

- Desenvolver um sistema de adoção com recomendação inteligente baseada no perfil do usuário.
- Facilitar o cadastro de animais disponíveis para adoção.
- Promover a visibilidade de ONGs e abrigos parceiros.
- Incentivar a adoção consciente por meio de informações e recomendações personalizadas.

2. Descrição do Projeto

Tema do Projeto:

Adopt.me - Sistema web de adoção de animais.

Problemas a Resolver:

- Dificuldade de encontrar animais compatíveis com o perfil do adotante.
- Falta de personalização nas plataformas de adoção.
- Processo de adoção pouco informatizado.

Limitações:

O sistema implementa um mecanismo de recomendação baseado em compatibilidade entre atributos previamente definidos, utilizando critérios objetivos como tipo de moradia, presença de crianças na residência, tempo disponível semanal e estilo de vida do adotante. A recomendação não substitui a decisão final do usuário, servindo apenas como mecanismo de apoio durante o processo de adoção. O sistema não considera fatores subjetivos, emocionais, aspectos legais do processo de adoção ou avaliações comportamentais mais complexas dos animais, limitando-se aos atributos cadastrados na plataforma.

3. Especificação Técnica

O sistema foi desenvolvido utilizando React no frontend e Flask (Python) no backend, permitindo a comunicação entre interface e serviços internos por meio de uma API REST. O mecanismo de recomendação foi implementado através do cálculo de compatibilidade entre o perfil do adotante e os atributos dos animais cadastrados, utilizando vetorização de dados e funções matemáticas da biblioteca Scikit-Learn para gerar recomendações personalizadas. O banco de dados principal utilizado é o PostgreSQL, responsável pelo armazenamento de usuários, animais e perfis cadastrados. O sistema conta ainda com autenticação via Google OAuth 2.0, autenticação baseada em JWT, infraestrutura containerizada com Docker e deploy automatizado em ambiente cloud.

3.1 Requisitos de Software

Requisitos Funcionais (RF)

RF001 – O sistema deve permitir o cadastro e autenticação de usuários.

RF002 – O sistema deve permitir que usuários cadastrem animais disponíveis para adoção.

RF003 – O sistema deve permitir que usuários visualizem animais disponíveis e iniciem o processo de adoção.

RF004 – O sistema deve disponibilizar filtros de busca por espécie, idade, porte e localização.

RF005 – O sistema deve recomendar animais com base na compatibilidade entre o perfil do adotante e os atributos dos animais cadastrados, utilizando cálculo de similaridade entre dados previamente definidos.

RF006 – O sistema deve exibir selo visual de compatibilidade nos animais considerados mais compatíveis com o perfil do adotante.

RF007 – O sistema deve permitir autenticação utilizando conta Google por meio do protocolo OAuth 2.0.

RF008 – O sistema deve apresentar interface visual baseada em cards contendo informações básicas sobre os animais cadastrados.

RF009 – O sistema deve armazenar dados de usuários, animais e perfis em banco de dados relacional.

RF010 – O sistema deve permitir edição e exclusão de cadastros por parte dos usuários responsáveis pelos anúncios.

RF011 – O sistema deve permitir que o usuário preencha um perfil contendo informações como moradia, presença de crianças, tempo disponível e estilo de vida para receber recomendações personalizadas.

RF012 – O sistema deve disponibilizar funcionalidades distintas para usuários interessados em adotar ou disponibilizar animais para adoção.

RF013 – O sistema deve calcular um percentual de compatibilidade entre o perfil do adotante e os animais cadastrados.

RF014 – O sistema deve destacar visualmente os animais com maior compatibilidade dentro da plataforma.

Requisitos Não Funcionais (RNF)

RNF001 – O sistema deve ter uma interface responsiva, adaptável a diferentes tamanhos de tela.

RNF002 – A autenticação deve ser segura, utilizando o protocolo OAuth 2.0.

RNF003 – O sistema deve processar recomendações de forma rápida.

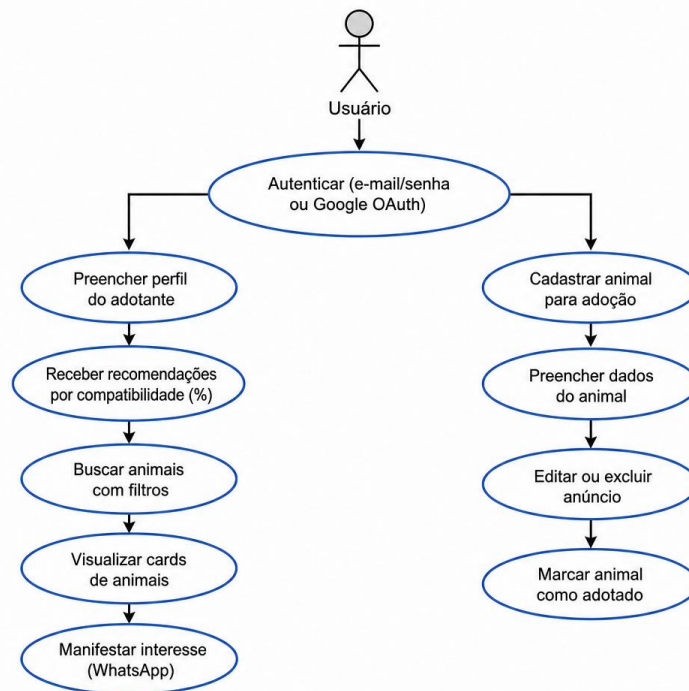
RNF004 – O sistema deve seguir boas práticas de arquitetura modular e escalável, com separação clara entre frontend (React), backend (Flask) e banco de dados (PostgreSQL).

RNF005 – O sistema deve utilizar HTTPS em produção para garantir a segurança das informações.

RNF006 – O sistema deve ser compatível com navegadores modernos (Chrome, Firefox, Edge).

RNF007 – A interface do sistema deve guiar o usuário por meio de fluxos distintos para adoção e doação, garantindo usabilidade intuitiva.

Representação dos requisitos:



3.2 Considerações de Design

O desenvolvimento do sistema foi estruturado seguindo princípios de separação de responsabilidades, modularização e organização em camadas, permitindo maior facilidade de manutenção e evolução futura da aplicação.

A arquitetura foi dividida em frontend, backend e banco de dados, separando a interface visual, a lógica de negócio e a persistência das informações. Essa organização permite desacoplamento entre componentes e facilita futuras melhorias no sistema.

O backend concentra a lógica principal da aplicação, incluindo autenticação, gerenciamento de dados e processamento do mecanismo de recomendação por compatibilidade.

Padrões adotados:

- **MVC** (Model-View-Controller) será usado de forma leve na estruturação do backend com Flask.
- **REST** será o padrão de comunicação entre frontend e backend.
- **OAuth 2.0** será usado como padrão de autenticação com o Google.

Modelo C4:

Contexto

O sistema permite interação entre usuários interessados em adotar ou disponibilizar animais para adoção, centralizando todo o processo em uma única plataforma.

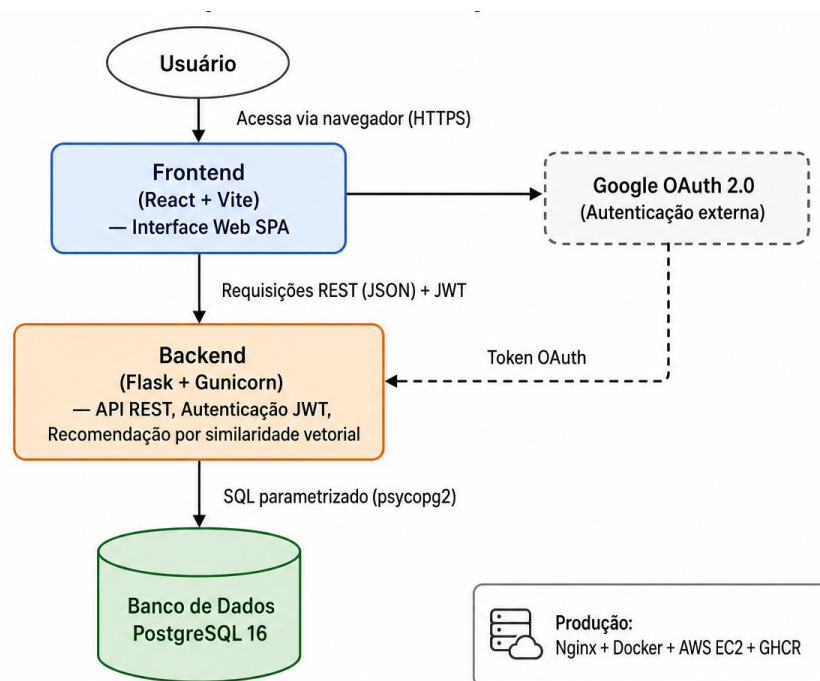
Usuários podem realizar autenticação, cadastrar animais, preencher perfil de adotante e receber recomendações personalizadas com base em compatibilidade entre atributos.

Container

A aplicação é composta pelos seguintes containers principais:

- Frontend Web Application (React)
- Backend API Application (Flask)
- Banco de Dados PostgreSQL
- Servidor Nginx responsável pelo gerenciamento das requisições externas

Além disso, o sistema realiza integração externa com o serviço de autenticação Google OAuth 2.0.



Componentes

Dentro do backend, o sistema é estruturado em componentes para manter a organização, escalabilidade e manutenção do código:

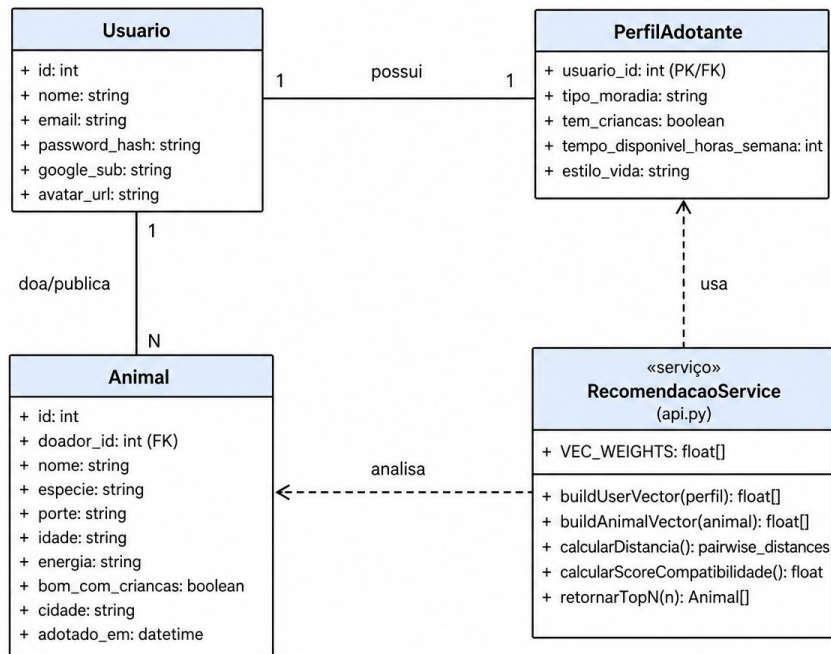
Principais componentes no Backend:

- Responsável pelo processamento central do sistema.
- Principais responsabilidades:
- API REST
- Autenticação de usuários
- Integração com Google OAuth
- Processamento da lógica de recomendação
- Validação e gerenciamento dos dados.

Principais componentes no Frontend:

- Responsável pela interface visual da plataforma e interação com o usuário.
- Principais funcionalidades:
- Exibição de animais cadastrados
- Sistema de filtros de busca
- Cadastro de animais
- Cadastro de perfil do adotante
- Exibição das recomendações personalizadas.

Diagrama de classe:



3.3 Stack Tecnológica

Linguagens de Programação:

- **JavaScript** – Utilizado no frontend (React).
- **Python** – Utilizado no desenvolvimento do backend e implementação da lógica de recomendação por compatibilidade.

Frameworks e Bibliotecas:

- **React** – Construção da interface do usuário.
- **Flask** – Framework utilizado no desenvolvimento da API backend.
- **Scikit-Learn** – Utilizado como biblioteca auxiliar para cálculo matemático de similaridade entre perfis.
- **Axios** – Comunicação HTTP entre frontend e backend.
- **Material UI** – Componentes visuais e estruturação da interface frontend.

Banco de Dados:

- **MySQL** – Localmente no desenvolvimento e migrado para nuvem no deploy (PostgreSQL).

Ferramentas de Desenvolvimento:

- **VSCode** – Editor de código.
- **Vite** – Inicialização e build do projeto React.

Ferramentas de Gestão de Projeto:

- **Microsoft Planner** – Organização de tarefas.
- **Figma** – Protótipos de interface.
- **GitHub** – Controle de versão.
- **Docker** – Containerização da aplicação e serviços

3.4 Considerações de Segurança

Proteção de dados:

O sistema utilizará **autenticação OAuth 2.0 com conta Google**, evitando o armazenamento de senhas sensíveis. Toda comunicação com o servidor será feita utilizando **HTTPS** para garantir a criptografia dos dados trafegados.

Riscos e mitigação:

- **Risco:** Invasão por força bruta → *Mitigação:* Login via Google com autenticação segura.
- **Risco:** Vazamento de dados sensíveis → *Mitigação:* Criptografia de tráfego (HTTPS) e boas práticas de segurança no backend.
- **Risco:** Ataques de injeção SQL → *Mitigação:* Uso de ORM ou prepared statements no backend.

4. Próximos Passos

Fase 1 – Portfólio I

- Levantamento detalhado dos requisitos junto do orientador.

- Criação dos protótipos de interface no Figma para validação visual.
- Desenvolvimento inicial do frontend com React.
- Implementação da autenticação com Google.

Fase 2 – Portfólio I I

- Finalização da interface.
- Implementação das regras de segurança.
- Testes e validações.
- Entrega e apresentação do TCC.

Considerações Professor(a) 1:

Assinatura: _____

Considerações Professor(a) 2:

Assinatura: _____

Considerações Professor(a) 3:

Assinatura: _____